

EXP.NO: 4 QUERYING LEDGER DATA WITH HYPERLEDGER FABRIC**AIM**

To query and retrieve ledger data from a Hyperledger Fabric blockchain network using JavaScript chaincode, and verify asset information stored in the world state database.

PROCEDURE

1. Navigate to the Hyperledger Fabric test network directory:

```
cd ~/fabric-dev/fabric-samples/test-network
```

2. Start the Fabric network and create the application channel:

```
./network.sh up createChannel -ca
```

3. Deploy the JavaScript Asset Transfer chaincode:

```
./network.sh deployCC -ccl javascript -ccn basic -ccp ../asset-transfer-basic/  
chaincode-javascript
```

4. Initialize the ledger with sample asset data:

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c  
'{"function":"InitLedger","Args":[]}'
```

5. Query all asset records stored in the ledger:

```
peer chaincode query -C mychannel -n basic -c '{"Args":["GetAllAssets"]}'
```

6. Query specific asset record (asset1):

```
peer chaincode query -C mychannel -n basic -c '{"Args":["ReadAsset","asset1"]}'
```

7. Create a new asset (asset20) and query state:

```
peer chaincode invoke -o localhost:7050 --ordererTLSHostnameOverride  
orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic -c  
'{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'
```

8. Query non-existing asset (asset50) to verify error handling:

```
peer chaincode query -C mychannel -n basic -c '{"Args":  
["ReadAsset","asset50"]}'
```

9. Stop the Fabric network:

```
./network.sh down
```

OUTPUT

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh up createChannel -ca
Creating channel 'mychannel'... done
Network Started Successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ ./network.sh deployCC -ccl javascript -ccn
basic -ccp ../asset-transfer-basic/chaincode-javascript
Chaincode committed successfully
```

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --
ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic
-c '{"function":"InitLedger","Args":[]}'
Ledger Initialized Successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic
-c '{"Args":["ReadAsset","asset1"]}'
{
  "ID": "asset1",
  "Color": "blue",
  "Size": 5,
  "Owner": "Tomoko",
  "AppraisedValue": 300
}
```

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode invoke -o localhost:7050 --
ordererTLSHostnameOverride orderer.example.com --tls --cafile $ORDERER_CA -C mychannel -n basic
-c '{"function":"CreateAsset","Args":["asset20","Green","8","Arun","1500"]}'
Asset asset20 created successfully

user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic
-c '{"Args":["ReadAsset","asset20"]}'
{
  "ID": "asset20",
  "Color": "Green",
  "Size": 8,
  "Owner": "Arun",
  "AppraisedValue": 1500
}
```

```
user@ubuntu:~/fabric-dev/fabric-samples/test-network$ peer chaincode query -C mychannel -n basic
-c '{"Args":["ReadAsset","asset50"]}'
Error: The asset asset50 does not exist.
```

RESULT

Hence the experiment was done successfully.